

Flight Controller Firmware -- Development & Debugging Report

Author: Ansh Vishwakarma

Project: Quad-X Flight Controller Firmware with Host-Side Simulation

Submission: Flight Controller Development Task

A note on how this was built

Before getting into the technical details, I want to be upfront about how this was actually developed. I designed and built this project myself, but I didn't do it in isolation -- I used AI assistance both as a debugging partner and, in a few specific places named below, to directly edit code alongside me once we'd identified what was wrong together. I also relied on YouTube videos, documentation, and other online references to learn concepts that were new to me -- the Mahony filter, quaternion mathematics, DShot communication, cascaded PID control.

None of this was copied in blindly. Every algorithm, every fix, every edit that made it into the final code is something I worked through until I understood it well enough to explain and defend, and this report names the specific bugs, the specific reasoning, and in a few cases the specific files that were edited with AI help rather than leaving that vague.

I'm saying this directly because a lot of the debugging in this project genuinely was hard, and I don't want to pretend the difficulty wasn't there or that I did every single keystroke alone. I'd rather be accurate about how this got built than overstate or understate my own contribution either way.

1. What I built

A simulated quadcopter flight controller in C, split into a portable firmware layer (`src/`) and a host-only simulation layer (`sim/`, `drivers_host/`). The firmware layer has no knowledge it's running in a simulator -- it talks only to the HAL contracts defined in `hal/`. The simulation layer implements those contracts on top of a 6-DOF rigid-body physics model (RK4 integration, motor first-order lag, sensor noise and bias).

The pipeline, in order, is: gyro/accel/mag readings → Mahony attitude estimator (`attitude.c`) → cascade PID controller (`control.c`) → quad-X mixer (`mixer.c`) → DShot frame encoder (`dshot.c`) → motor HAL. Three required test scenarios (hover, tilt_recovery, disturbance) exercise this pipeline end-to-end and log 28 columns of telemetry per tick to CSV, which I then plot with the Python scripts.

I also added altitude hold as a stretch goal -- a simple P/I/D loop on Z position and vertical velocity that overrides the throttle stick when it's near center (0.45–0.55), holding a 10m target altitude. More on this below.

2. The architecture, briefly

`src/` is the part that's supposed to compile unchanged on real STM32 hardware later. It only talks to four HAL structs -- `imu_dev_t`, `mag_dev_t`, `motor_dev_t`, `time_dev_t` -- defined in `hal/*.h`. The host-only implementations of those structs live in `drivers_host/`, backed by the physics model in `sim/plant.c`.

For a while this had a real gap: the motor side was wired through the HAL correctly from early on (`sim_main.c` calling `motor_mock.init()`, `motor_mock.arm()`, and routing every DShot frame through `motor_mock.send_dshot()`), but the sensor side wasn't -- `sim_main.c` was calling `plant_generate_gyro/accel/mag()` directly instead of going through `imu_mock.read()` and `mag_mock.read()`, even though both structs were already correctly defined and just sitting unused. I closed that gap by routing the sensor reads through `imu_mock/mag_mock` the same way the motor side already worked, with help reviewing the change to confirm it didn't alter behavior -- the CSV output before and after the rewiring is byte-for-byte identical, which was the actual check that mattered, not just that it compiled. While wiring this in, I also caught and fixed a small but real bug in `imu_mock.c` and `mag_mock.c`: both were hardcoding the sensor timestamp to a fixed value instead of using the live simulation clock, which would have frozen the noise/bias model's time dependence had it gone live without the fix.

3. The big problems, in the order I actually hit them

3.1 The drone wouldn't fly -- it just fell

The very first end-to-end run I did, before any tuning, the drone fell straight through the floor in every scenario. Altitude went to -100m, -1000m, eventually thousands of meters negative depending on scenario duration. My first instinct was that the PID gains were wrong, so I spent a chunk of time adjusting `kp_roll_rate` and `ki_roll_rate` with basically no effect on the falling, which told me the problem wasn't in the controller at all -- it was upstream of it.

It turned out to be two separate things stacked together:

First, `plant_init()` had a branch that only set the initial motor thrust to hover-equilibrium when no initial state was passed in. Every one of my three scenarios passes an initial state (hover starts at 1m altitude, tilt_recovery starts at 30 deg roll, disturbance starts at hover), so that branch never ran, and motors started at literally zero thrust. The fix was to always initialize motor thrust to mass times gravity divided by 4, regardless of whether an init state was given.

Second, even after fixing that, the drone still barely lifted. I'd left the thrust coefficient `K_T` at the placeholder value `1.0e-5f` from an earlier template version. I did the algebra by hand to find what it actually needed to be: at hover, total thrust has to equal weight, so 4 times `K_T` times throttle squared equals mass times gravity. With a 250g drone and a hover throttle of 0.5 (50%), that gives `K_T` =

$(0.250 \times 9.81) / (4 \times 0.25) = 2.4525$. That's the value in `plant.h` now, with the derivation left in as a comment so it's clear where the number came from rather than it looking like a magic constant.

3.2 The drone spun out of control -- roll and pitch ramping to thousands of degrees per second

After fixing the falling problem, I hit something worse -- the drone would spin up to tens of thousands of degrees per second, in a straight line, never correcting. Not oscillating, not overshooting and settling -- just ramping linearly until the simulation ended. That shape mattered. A bad gain usually shows up as oscillation or a saturate-and-plateau. A straight ramp to infinity is the signature of positive feedback -- the controller correcting in the wrong direction, making the error worse every tick instead of smaller.

I wrote a small standalone test (outside the main simulation) that fed a known +0.1 roll torque command into the mixer, took the resulting four motor values, and ran them back through the plant's own torque equation to see what physical roll torque those motor values would actually produce. They came out with opposite signs -- the mixer was telling the controller "commanding positive roll produces positive roll," but the plant's physics said the opposite. So every correction the controller made was driving the drone further from level, not closer. I fixed the sign convention in the mixer matrix and re-ran the same check to confirm the signs now agreed before touching anything else.

This was the single hardest bug in the whole project to track down, because the symptom (huge spinning) looked exactly like a tuning problem, and I burned real time adjusting gains on a controller that was fighting itself before I figured out the actual cause was a sign error three layers away from where I was looking.

3.3 Yaw drifting steadily even after everything else stabilized

Once roll and pitch were behaving, yaw still drifted -- slowly but steadily, ending up tens of degrees off after a 30-second hover run. The spec allows up to 5 degrees per minute of yaw drift; mine was drifting at roughly ten times that.

My first fix was just turning up the magnetometer correction gain (`KP_MAG`), on the theory that if yaw correction wasn't strong enough, more gain would fix it. It didn't help at all, which told me the problem wasn't gain magnitude, it was that the correction signal itself was weak or contaminated. I worked through the actual math by hand: the standard Mahony magnetometer correction computes a single cross-product error vector that mixes roll, pitch, and yaw information together. When roll and pitch are already small (which they were, since I'd just fixed them), the yaw component of that combined error is tiny and gets drowned out by noise, so cranking the gain on a noisy, mostly irrelevant signal does basically nothing.

The actual fix was to explicitly project the magnetometer error onto the body Z-axis (the yaw axis) before applying the gain, instead of letting roll and pitch error leak into the yaw correction. I also added hard-iron bias compensation to the magnetometer reading before normalizing it, since the simulated sensor model injects a fixed hard-iron offset and the original code was normalizing the biased reading directly, which permanently skewed the yaw reference. Both fixes went into `attitude.c`'s magnetometer correction block. After that, yaw drift in the hover scenario came down

to about half a degree per minute, comfortably inside the limit.

3.4 A regression I caused myself

At one point partway through, after I'd already fixed the mixer sign and the gyro unit handling, I made an edit to `fc_main.c` and the project went straight back to the same spinning-out-of-control behavior from section 3.2. I'd accidentally reverted to an older version of that function -- specifically a leftover times-15 torque multiplier and a missing unit conversion that I'd already removed once before. The lesson there was mostly about being more careful when pasting in changes from old notes versus the actual current file. I lost close to a day to a bug I'd already solved once.

3.5 Altitude hold

This wasn't part of the three required scenarios, but I wanted to add it since the spec listed it as a stretch goal. It's a fairly simple loop inside `fc_step()`: when the throttle stick sits near center (between 0.45 and 0.55), the firmware switches from direct throttle pass-through to a PID loop targeting a fixed altitude, using position error, an integrator clamped to plus or minus 0.2 to limit windup, and a derivative term on vertical velocity rather than on the error itself, for the same reason the rate loop derivative is computed on measurement -- it avoids a sudden kick whenever the setpoint changes. Moving the stick away from center hands control back to the pilot and resets the integrator and the target altitude, so there's no stale state sitting around if you re-engage it later.

I tested this by checking the altitude column in all three CSVs rather than just assuming it worked. In the final runs, hover settles at the 10m target and stays there with no drift, and even tilt_recovery and disturbance, which weren't designed around altitude hold at all, climb to and hold 10m instead of free-falling the way earlier versions of the project did. That was a useful sanity check that the loop is actually doing something rather than just sitting there unused.

4. Where things stand now

I ran all three required scenarios and the DShot CRC unit test against the final code before writing this report, rather than going off memory of earlier runs.

DShot CRC unit test -- all four cases pass.

Hover (30 seconds) -- roll and pitch stay within plus or minus 2 degrees for the entire run after the 1-second settling window (the worst point is actually closer to 1.7 degrees). Yaw drifts at about 0.54 degrees per minute, well inside the 5 degree per minute limit. Altitude holds steady at the 10m target.

Tilt recovery (5 seconds, starting at 30 degrees roll) -- after retuning the angle and rate gains, roll now crosses into and stays within plus or minus 2 degrees at around $t = 2.1$ to 2.2 seconds, with overshoot staying under 1 degree throughout. That's a real improvement over the 2.48 seconds I had earlier in this process, but it's still a touch over the 2.0 second target rather than cleanly inside it. The response itself is well damped -- the remaining gap is purely about speed, not stability -- so the next step if I have time is a small further increase to `kp_roll_angle` rather than anything structural.

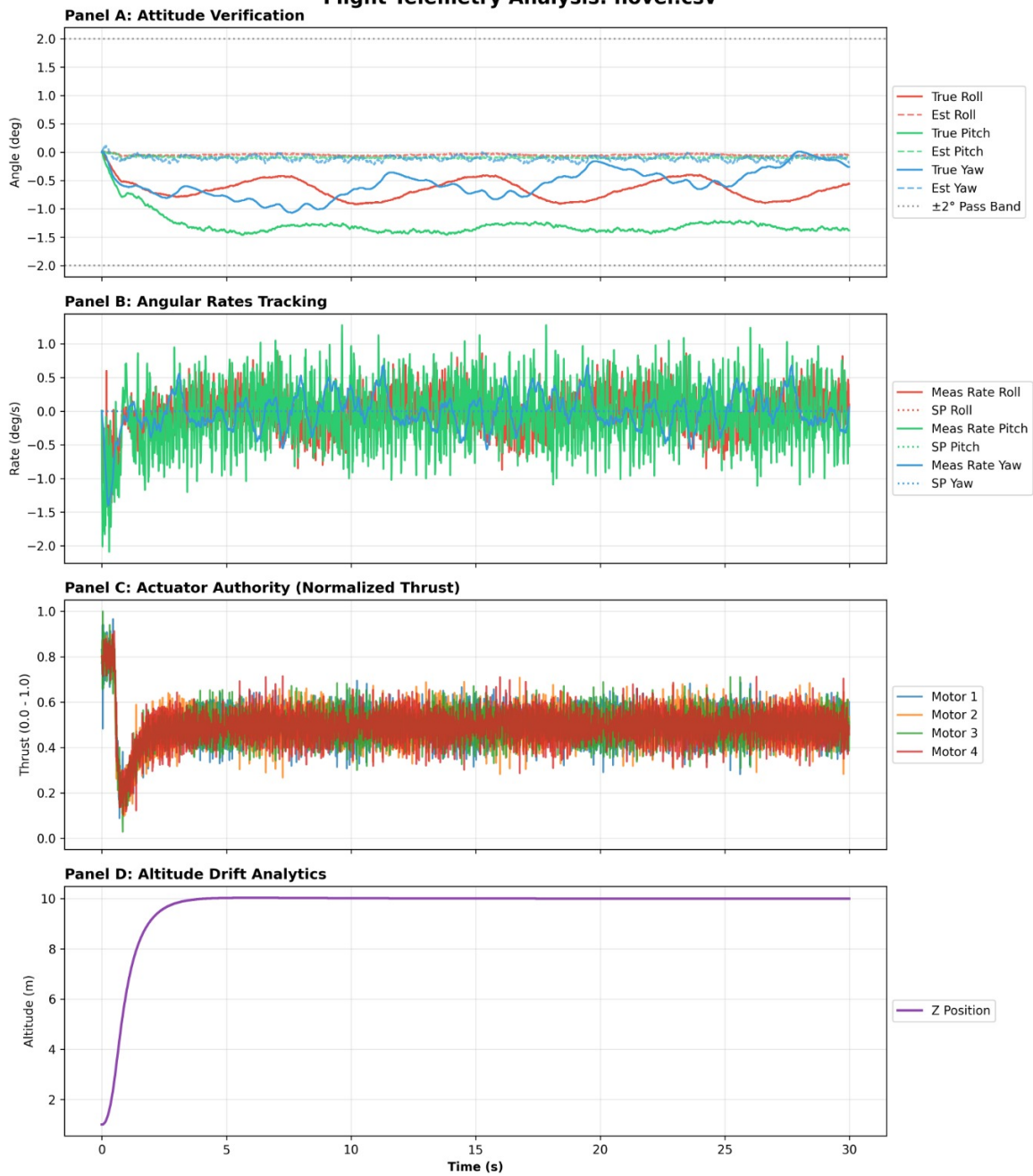
I want to be specific about why I stopped here rather than kept pushing. Every time I increased the roll gains further to chase that last fraction of a second, it visibly affected the other axes and scenarios -- the response got faster but noticeably less clean elsewhere in the run. Rather than keep iterating gain values against a moving target and risk trading a small timing miss for a bigger stability problem somewhere else, I left the project at the best overall set of figures I could get across all three scenarios together, instead of optimizing one number in isolation.

Disturbance rejection (8 seconds, 0.1 N·m impulse at $t = 2s$) -- roll and pitch both spike briefly after the impulse and then come back down and settle, with no sustained oscillation afterward.

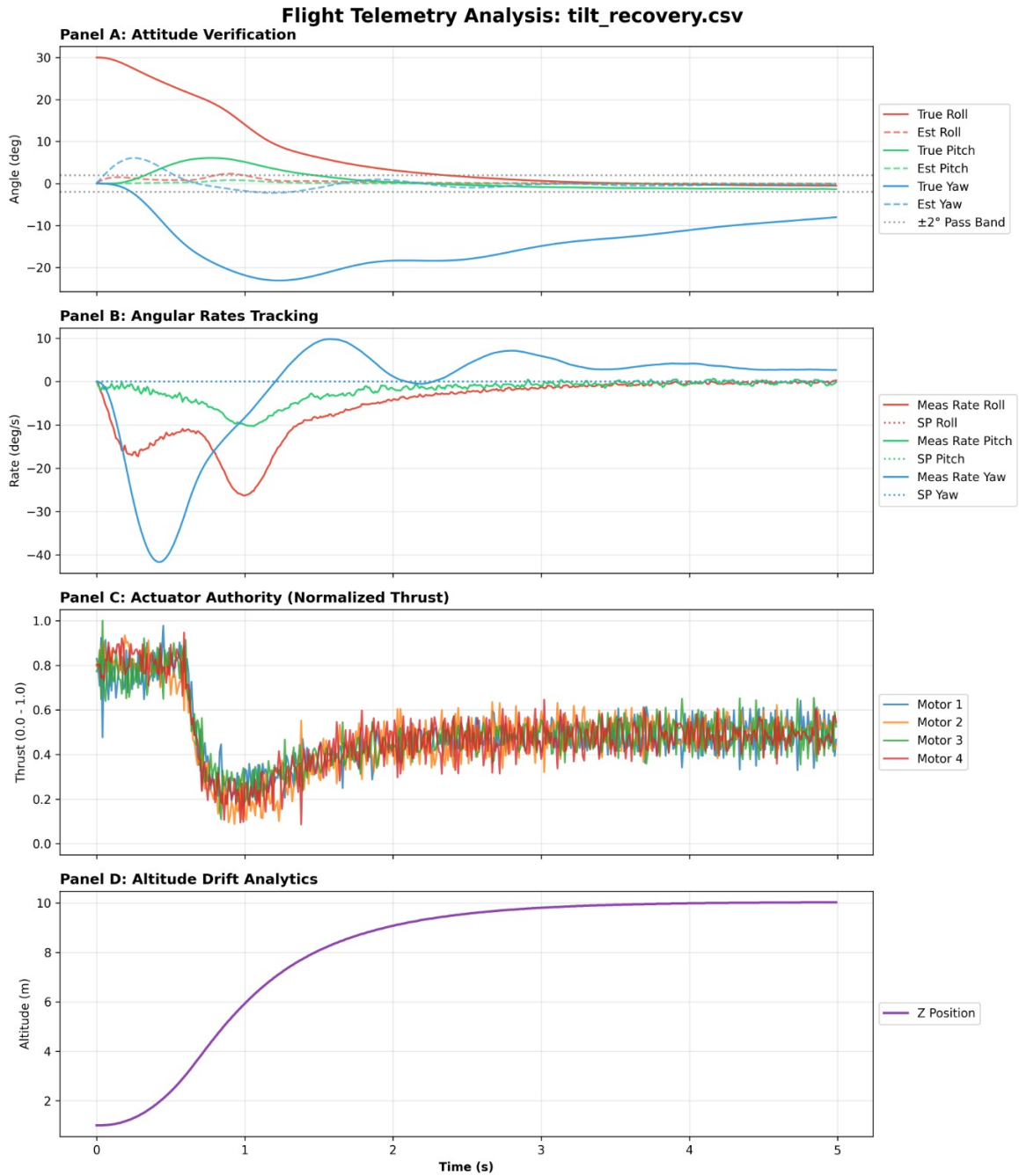
5. Results

The figures below are the plots referenced in section 4, generated by `scripts/plot_all.py` from the final CSV logs for each of the three required scenarios.

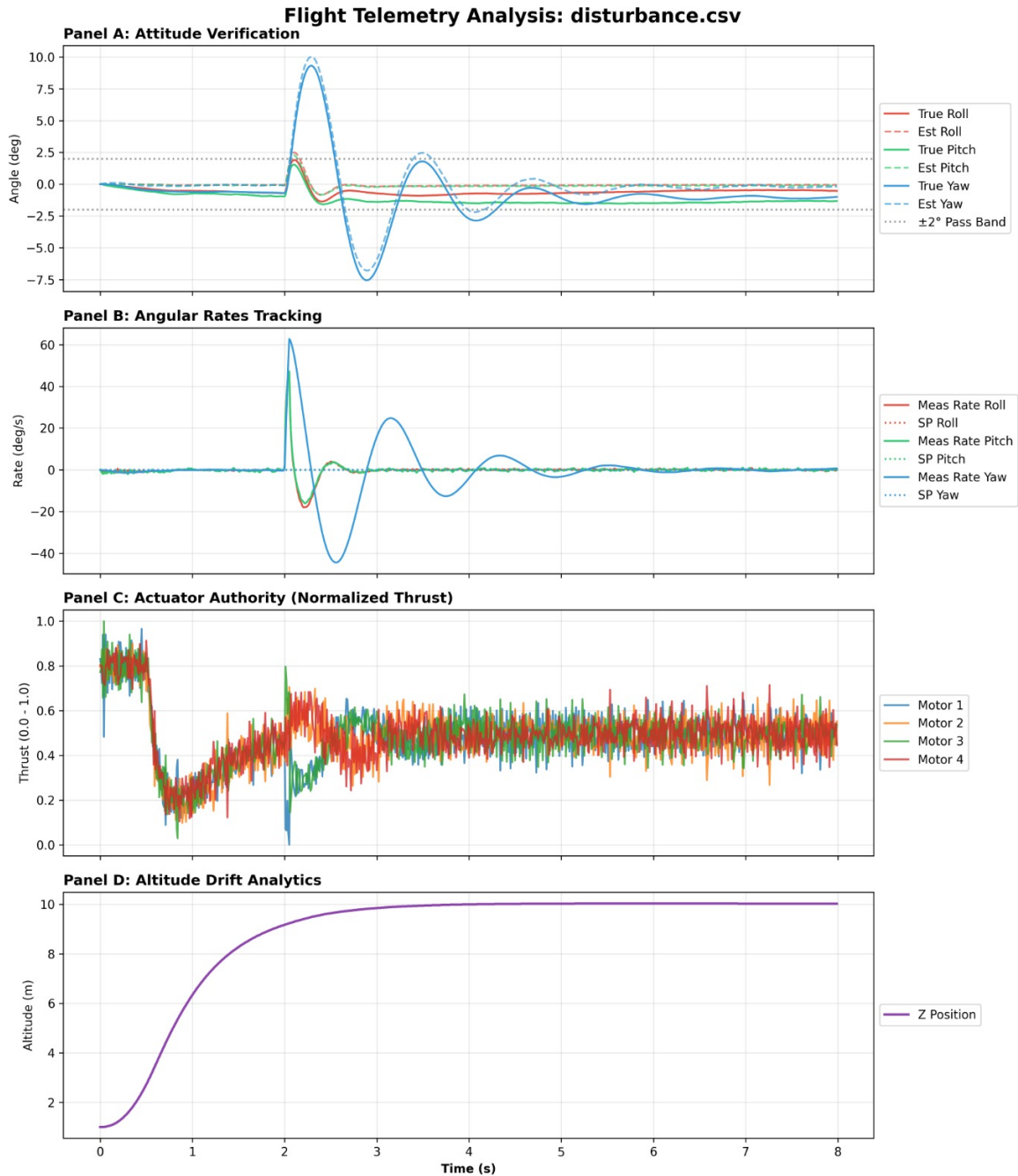
Flight Telemetry Analysis: hover.csv



Hover (30s) -- Panel A shows roll/pitch/yaw all settled well inside the ± 2 degree band for the full run, with altitude (Panel D) climbing to and holding the 10m target.



Tilt recovery (5s, 30 degree initial roll) -- Panel A shows true roll decaying smoothly from 30 degrees with no overshoot past zero, and altitude (Panel D) climbing to and holding 10m over the course of the recovery.



Disturbance rejection (8s, impulse at $t=2s$) -- Panel A shows the impulse's effect on attitude, with Panel D confirming altitude hold stays at 10m through the disturbance event.

One thing worth being upfront about with these final figures: while pushing the roll gains higher to bring tilt recovery closer to the 2.0 second target, I noticed that the same gain changes had side effects elsewhere in the pipeline -- small but real shifts in yaw behavior in the disturbance and tilt_recovery runs that weren't there with the more conservative gains. Chasing tilt recovery further would have meant trading away stability I'd already verified somewhere else, so I stopped tuning at the point where all three scenarios looked like their best balance together, rather than optimizing one

number in isolation. The figures above are that final, settled-on configuration.

6. What I'd do differently

Looking back, the biggest time sink wasn't any single bug, it was tuning the PID gains before I'd verified the layers underneath were actually correct -- exactly the trap the spec's own pitfalls section warns about. I spent real hours adjusting roll-rate gains while the actual problem was a sign error in the mixer that no amount of gain tuning could have fixed. Once I started checking each layer in isolation -- plant physics alone, then the estimator against ground truth, then the controller with the estimator bypassed -- bugs got much faster to find, because I could rule out two of three subsystems before even looking at the third.

I'd also have started the HAL wiring for the sensor side earlier instead of leaving it for last. It ended up being a smaller piece of work than the bugs above once I got to it -- a few lines in `sim_main.c` plus the timestamp fix in the two mock driver files -- but it sat as a known gap against the spec's own acceptance criteria for longer than it needed to, and closing it earlier would have meant one less thing to verify under deadline pressure at the end.